Pacing Gain and Congestion-Window Growth: Packet Models and SCP Experiments

Skai Uijing
Independent Researcher
China
skaiuijing@gmail.com

Abstract

Pacing changes when new congestion-window credit becomes a transmission and, consequently, when it produces an ACK. This study began with slower startup after adding pacing to a TCP-like transport, SCP. We express slow-start growth through actual transmissions, construct Fibonacci and immediate-send comparison models, and solve an explicitly defined packet scheduler with immutable pacing reservations. Its asymptotic window multiplier is $\min(e^{g/2}, 2)$, with a transition at $g = 2\ln 2$. We then examine 66 transfers through the actual SCP C implementation and a fixed-feedback-delay virtual network. At seven low gains, late measured multipliers differ from the queued-model prediction by less than 0.01%. Near the ideal transition, millisecond execution and empty-queue reset change the phase and the finite transition. A separate 66-transfer congestion-avoidance campaign isolates ACK-count growth from transmission timing: integer-byte updates produce a 15.719% final full-packet window deficit relative to the real-valued ACK recurrence. In both phases, higher gain substantially concentrates sending while completion time improves little. The results distinguish growth in time, ACK-indexed growth, and within-RTT transmission span.

CCS Concepts

• **Networks** → **Transport protocols**.

Keywords

pacing, slow start, congestion avoidance, congestion window, virtual-network experiments

1 Introduction

While building SCP with Reno-style congestion control, I first used burst sending and then added pacing. Transfers slowed down even though the ratio of useful data to total transmitted data barely changed. Tracing packets and ACKs suggested a timing question: an ACK can enlarge the window immediately, but newly permitted packets still have to pass through the pacer before they produce feedback.

For example, send one packet at time zero. Its ACK returns at R, increasing the window to two packets. If the next packets leave at R and $1.5R$, only one has been acknowledged at $2R$: the window is three, not four. All window updates occur on ACK arrival. This example motivates the analysis; it does not describe every pacer or imply that pacing generally harms network performance.

We also isolate loss-free congestion avoidance from one MSS to examine how its smaller ACK increments and integer-byte storage change this timing problem.

2 ACK accounting and two comparison models

Assume a fixed send-to-ACK delay R, no loss, continuously available data, one ACK per full-sized packet, and indefinite slow start. Each ACK adds one MSS, denoted M, to the window. Normalize time and window as $\tau = t/R$ and $w = \mathrm{cwnd}/M$. With actual packet-send times s_j in normalized units, including $s_0 = 0$,

$$\begin{aligned} w(\tau) &= 1 + \sum_{j\ge 0} \mathbf{1}_{\{s_j+1\le\tau\}}, \\ W_{n+2} &= W_{n+1} + S_n. \end{aligned} \tag{1}$$

Here $W_n = w(n)$ is sampled after all boundary events, and S_n counts sends in $(n, n+1]$. Sampling at RTT boundaries does not defer ACK processing.

Immediate sending. ACKs in $(n, n+1]$ can only acknowledge packets already in flight just after n. If their number is $\mathcal{F}_n$, then $W_{n+1} - W_n \le \mathcal{F}_n \le W_n$. Thus $W_n \le 2^n$. Sending both newly permitted packets immediately at every ACK attains $\mathrm{cwnd}(nR) = M2^n$ under these ideal assumptions.

Fixed batches. A separate comparator reads its window V_n at each boundary and sends at $n + i/V_n$, for $i = 0, \ldots, V_n - 1$. ACKs update the window immediately but leave that batch unchanged. The next boundary supplies the next batch's first packet, so $S_n = V_n$ and

$$\begin{aligned} V_{n+2} &= V_{n+1} + V_n, \quad V_0 = 1,\ V_1 = 2, \\ \mathrm{cwnd}_{\mathrm{fixed}}(nR) &= MF_{n+2} = \frac{M(\phi^{n+2} - \psi^{n+2})}{\sqrt{5}}. \end{aligned} \tag{2}$$

where $\phi = (1+\sqrt{5})/2$, $\psi = (1-\sqrt{5})/2$, and $F_0 = 0, F_1 = 1$. This gives $1, 2, 3, 5, 8, 13, \ldots$. It is a deliberately simplified comparator. For any scheduler satisfying the additional service condition $S_n \ge W_n$, induction in (1) gives $MF_{n+2} \le \mathrm{cwnd}(nR) \le M2^n$. The lower condition requires justification for the scheduler being studied; earlier reservation alone is not a proof. Appendix A proves the lower comparison for the specific queued reference below by comparing actual packet times. It is not assumed for every pacer. These bounds motivate exponential growth but do not establish an exact exponential solution.

3 An exact queued reference model

3.1 Reservations, flight, and feedback

Consider the following particular scheduler, with a fixed gain $g \ge 1$. After ACK k, the window is $k+1$. The ACK releases one flight position and adds one window position, permitting two new packets. Both are appended to the reservation queue with spacing $1/\lfloor g(k+1)\rfloor$; existing timestamps remain unchanged. Let a_k be ACK time, b_k the new batch's first send, and T_k the next available slot after

this batch. Then

$$\begin{aligned} a_k &= s_{k-1} + 1, & b_k &= \max(a_k, T_{k-1}), \\ s_{2k-1} &= b_k, & s_{2k} &= b_k + \frac{1}{g(k+1)}, \\ T_k &= b_k + \frac{2}{g(k+1)}. \end{aligned} \tag{3}$$

Initialize $s_0 = 0$, $T_0 = 1/g$. Actual sends move packets from the reserved queue to flight. Consequently $F+Q = w$ after each ACK's reservations, and actual flight respects the window. This model retains the spacing after the last reserved packet, even if the queue becomes empty. Section 4 explains why the tested SCP implementation differs.

3.2 Solving the event recurrence

Write $c = 2/g$, $H_k = \sum_{i=1}^{k} 1/i$, and $D(q) = H_{2q} - H_q$. Group ACK indices as $q \le k < 2q$, where $q = 2^m$, and put $B_m = b_{2^m}$. These are analytical groups, not batches processed by the implementation. The exact solution is

$$B_{m+1} = B_m + \max\{1, cD(2^m)\}, \quad B_0 = 1, \tag{4}$$

$$b_k = B_m + c(H_k - H_{2^m}), \quad 2^m \le k < 2^{m+1}. \tag{5}$$

To verify it, note that $b_k = \max(a_k, b_{k-1} + c/k)$ for $k \ge 2$, and $a_{2j} = b_j + 1$, $a_{2j+1} = b_j + 1 + c/[2(j+1)]$. At the first index q of a group, the two candidate times are $B_{m-1} + 1$ and $B_{m-1} + cD(q/2)$, proving (4). Set $\Delta = B_m - B_{m-1} \ge 1$. At interior even indices,

$$T_{2j-1} - a_{2j} = \Delta - 1 + c[D(j) - D(q/2)] > 0. \tag{6}$$

At interior odd indices the bracket additionally contains $1/(2j+1) - 1/(2j+2) > 0$. Since $D(j+1) - D(j) = 1/[(2j+1)(2j+2)] > 0$, the queue branch wins throughout each group's interior. Summing its increments proves (5) by induction. In particular,

$$b_k = cH_k + 1 - c + \sum_{i=0}^{\lfloor \log_2 k \rfloor - 1} \left[1 - cD(2^i)\right]_+ . \tag{7}$$

Here $[x]_+ = \max(x, 0)$. Together with (3) and (1), this determines the window at every time without fitting a growth curve.

3.3 Gain, finite spans, and the limiting multiplier

Group m contains packets $2q-1, \ldots, 4q-2$. Its first-to-last send span Λ_m and extra ACK waiting time I_m, excluding the nominal trailing spacing, are

$$\frac{\Lambda_m}{R} = \frac{2D(q) - 1/(2q)}{g}, \qquad \frac{I_m}{R} = \left[1 - \frac{2D(q)}{g}\right]_+ . \tag{8}$$

Because $D(q) \uparrow \ln 2$, the limiting transition is $g_c = 2\ln 2 \approx 1.38629436112$. Below it, extra waiting eventually ends, $b_k = (2/g)H_k$ + constant, and $w(\tau) = A_g e^{g\tau/2} + O(1)$. At or above it, $B_m = m + 1$. Counting sends gives, for integer $n \ge 1$ and $0 \le u < 1$,

$$\begin{aligned} w(n+u) &= 2^{n-1} \min(e^{gu/2}, 2) + O_g(1), \\ r(g) &:= \lim_{n\to\infty} \frac{W_{n+1}}{W_n} = \min(e^{g/2}, 2). \end{aligned} \tag{9}$$

For finite $g \ge g_c$, the boundary value is exactly $W_n = 1 + 2^{n-1}$, rather than 2^n. The group span approaches $\Lambda_m/R \to g_c/g$: higher gain compresses the active part of each group. This is an asymptotic statement about packet-index groups, not a claim that every fixed RTT interval has this span.

4 From the reference model to an SCP experiment

4.1 What an instantaneous-rate approximation predicts

A different idealization updates the entire sending rate continuously: in normalized units $x(\tau) = gw(\tau)$, while window credit remains available. Fixed feedback gives $w'(\tau) = x(\tau-1) = gw(\tau-1)$. Substitution of $w = Ae^{\lambda\tau}$ yields

$$\begin{aligned} \lambda e^{\lambda} &= g, \\ F(\tau) &= \int_{\tau-1}^{\tau} x(v)\, dv = (e^{\lambda} - 1) w(\tau). \end{aligned} \tag{10}$$

The flight constraint requires $e^{\lambda} \le 2$, reaching equality at $g = 2\ln 2$. Thus the two idealizations share a transition value but differ below it. At $g = 1$, the instantaneous-rate multiplier is approximately 1.763223, while the immutable-queue multiplier is $\sqrt{e} \approx 1.648721$. Previously reserved packets retain older, wider spacings in the latter model. An exponential ansatz also fixes a compatible history; it does not determine the amplitude generated by a single initial packet.

4.2 Integer scheduling in the tested core

The experiment executes the SCP C sender, receiver, pacing callback, and ACK processing. The virtual network transports their packets; it does not compute the window from a mathematical recurrence. The tested core stores reservation times in unsigned 64-bit Q32.32 milliseconds. For payload length ℓ, measured RTT R_{ms}, window C in bytes, and compile-time integer reciprocal G, the spacing is

$$\begin{aligned} d_{\mathrm{Q32}} &= \left\lfloor \frac{\ell R_{\mathrm{ms}} G}{C} \right\rfloor, \\ G &\simeq 2^{32}/g, \qquad g_{\mathrm{effective}} = 2^{32}/G. \end{aligned} \tag{11}$$

An appended packet adds this spacing to the last reservation. Fractional parts accumulate while the queue persists, instead of truncating each spacing to whole milliseconds. The send routine uses the reservation's integer part: a packet reserved for 1.8 ms is eligible at 1 ms. The virtual clock advances by 1 ms, allowing multiple packets in a tick. ACK-triggered scheduling and the pacing timer share the due-send routine.

When the queue is empty, its first new packet is assigned the current time. The previous queue's trailing interval is not retained. This reset and the execution grid are intentional implementation rules. They distinguish the experiment from (3), particularly when the interval falls below a millisecond. The absence of SRTT uses the core's 1 ms fallback; established feedback supplies the measured RTT.

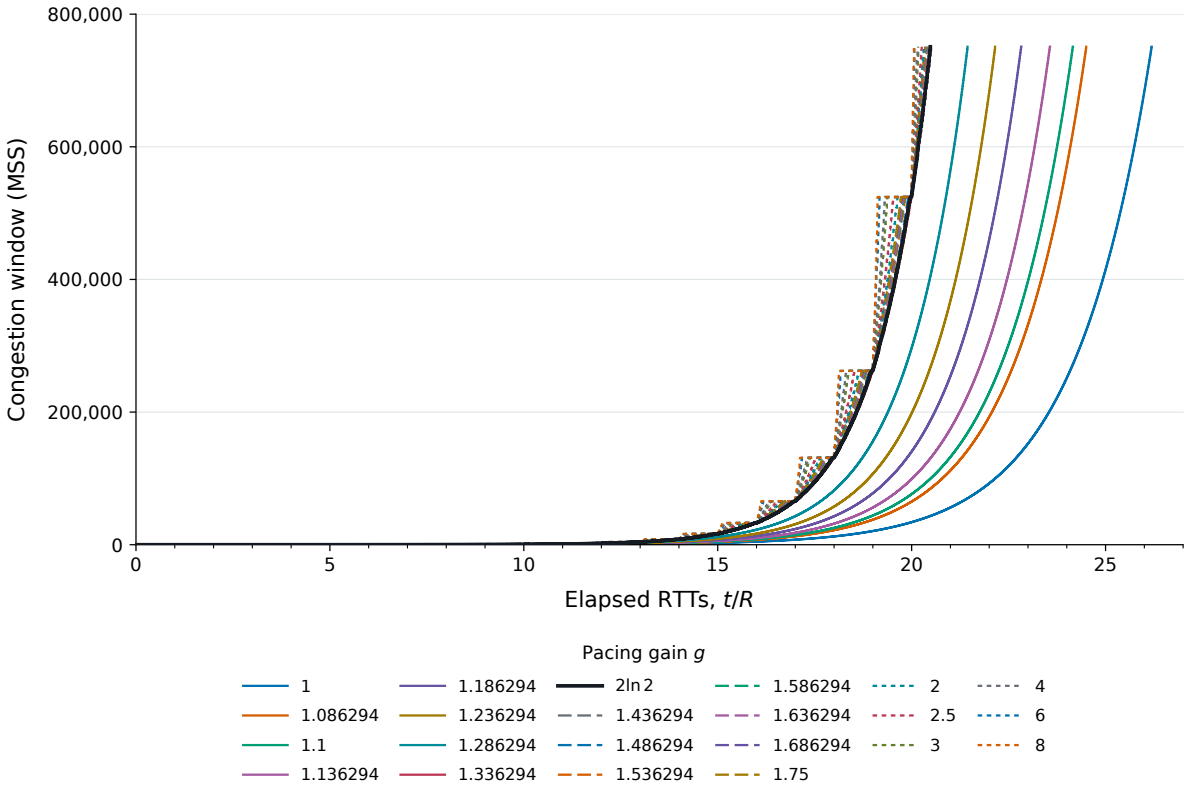


Figure 1: Congestion-window evolution from actual SCP core logs, for all 22 gains. The horizontal coordinate is elapsed time divided by the 100 ms feedback RTT; the window axis is linear in MSS. Curves retain the measured within-RTT evolution. Overlap is an experimental result, not an omitted gain.

Table 1: SCP virtual-network experiment configuration.

Quantity	Configuration
Data and packet size	1 GiB generated data; MSS 1430 bytes; final payload 584 bytes
Window and mode	Initial window 1 MSS; sustained slow start
Network	50 ms per direction; no loss, rate cap, or serialization delay
Feedback RTT	100 ms from actual DATA send to returning ACK input
Observation	Core state every 1 ms; actual DATA and ACK events logged
Gains	$g_c + 0.05k$, $k = -6, \ldots, 6$; 1, 1.1, 1.75, 2, 2.5, 3, 4, 6, 8
Repetitions	22 gains, three deterministic repetitions each
Limits	cwnd cap $2^{31} - 1$ bytes; buffer/window settings 1 GiB

4.3 Configuration and verification

All 66 transfers completed with 750,869 DATA packets and matching endpoint checksums. No loss, retransmission, or cwnd-cap contact was observed. Fifteen functional tests passed. The shared core, library, network, and common-test files matched the reviewed baseline in the recorded audit. Campaign records and repeat hashes were checked across all runs; the critical-gain first run also received independent packet-level reprocessing.

At the target g_c, $G = 3098164009$, so the effective gain is 1.386294361280859. The first DATA transmission defines time zero; the preceding handshake is excluded. The three repetitions reproduce identical traces, so no random-error bars are appropriate. Finite data availability affects the final intervals and is marked separately in the span figure.

5 Measured congestion-window growth

The low-gain regime. Figure 1 shows slower growth at low gain. For the seven sampled gains from 1 through $g_c - 0.10$, the measured late boundary ratio W_{18}/W_{17} has less than 0.01% relative error against $e^{g_{\text{effective}}/2}$. For example, at $g = 1$ it is 1.648883. This is close agreement with the queued reference and a clear distinction from the instantaneous-rate prediction 1.763223. It supports that reference's growth multiplier in this regime, without asserting equality of every timestamp or window value.

Doubling does not remove the startup deficit. At the critical target, the first eight boundary windows are 1, 2, 3, 5, 9, 17, 33, 65

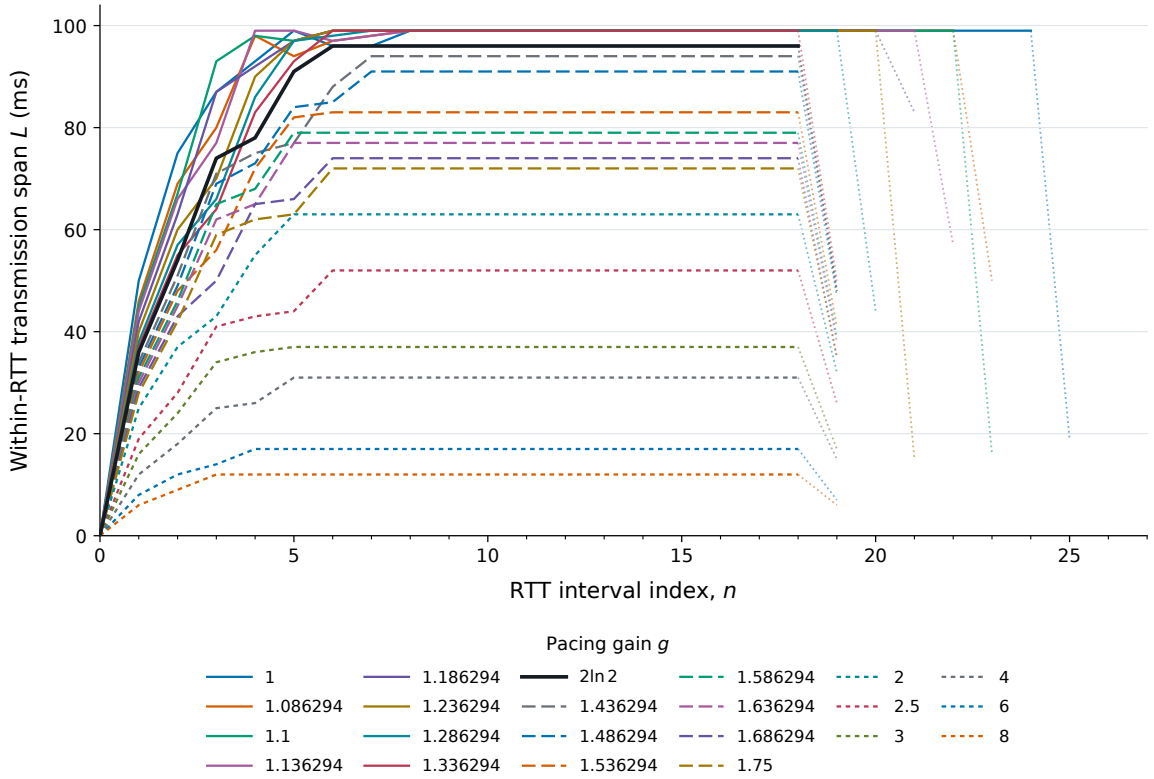


Figure 2: First-to-last actual DATA transmission span in each fixed RTT interval, for all gains. The faint dotted final segments indicate intervals shortened by exhaustion of the finite transfer. The spans come from packet timestamps; they are not calculated by dividing a window by a nominal rate.

Table 2: Measured growth and completion for selected gains.

Target gain	W_{18}/W_{17}	W_{18} (MSS)	Last ACK (t/R)
1	1.648883	12,548	26.19
$g_c - 0.10$	1.902493	82,260	21.44
$g_c - 0.05$	2.000000	132,096	20.50
g_c	2.000000	132,096	20.48
2	2.000000	133,120	20.32
8	2.000000	139,264	20.06

MSS. The window reaches 129 MSS at RTT 8, after which the measured samples satisfy

$$W_n = 129\,2^{n-8}, \qquad 8 \le n \le 20. \tag{12}$$

The multiplier is exactly two in this range, but $W_n/2^n = 129/256$: the window is still 49.609375% below the immediate-send reference. Earlier timing differences persist as an amplitude difference. At all observed 1 ms samples from 800 through 1899 ms, this run also satisfies $w(\tau+1) = 2w(\tau)$. This finite trace demonstrates repetition of the internal phase, not merely equality at RTT boundaries. It does not prove an infinite-time identity or a universal transition for other implementations.

6 Within-RTT sending span and concentration

For this figure, let $\mathcal{P}_n$ contain sends in the half-open interval $[nR, (n+1)R)$, measured from the first DATA transmission. Define

$$L_n = \max_{j\in\mathcal{P}_n} t_j - \min_{j\in\mathcal{P}_n} t_j. \tag{13}$$

A singleton has span zero; an empty interval has no defined span. This bin convention differs from the right-closed send-count intervals in (1). It also differs from the packet-index groups defining Λ_m in (8). These measurements cannot be interchanged near a phase boundary.

Evolution within one run. At the critical target, L_n initially takes values 0, 36, 54, 74, 78, 91 ms for $n = 0, \ldots, 5$, then remains 96 ms for $n = 6, \ldots, 18$. The next interval has span 48 ms because the finite transfer ends. The measured plateau is reached in finitely many rounds. By contrast, the exact reference group's span approaches its limit through harmonic corrections.

Variation across gains. At RTT 18, the span is 99 ms for the sampled gains through $g_c - 0.05$, then 96 ms at g_c, 63 ms at $g = 2$, and 12 ms at $g = 8$. A 100 ms interval sampled on a 1 ms grid has a largest possible first-to-last span of 99 ms. A 99 ms result therefore means sends reach both edge bins, not that the pacer fills every instant or even every bin.

Why the execution rule matters. Near g_c, the nominal full-packet spacing falls below 1 ms at a window of 73 MSS. Fractional accumulation prevents a continuously queued stream from becoming unlimited-rate simply through per-packet truncation. However, empty-queue resets can still place newly permitted packets in the current millisecond. The critical trace exhibits no reserved backlog at the 1 ms samples from 800 through 1899 ms; its repeating ACK phases can consequently reproduce a doubled transmission pattern on the grid.

From g_c to $g = 8$, the last DATA ACK moves from 2048 to 2006 ms, an improvement of about 2.05%. In the same comparison, L_{18} falls eightfold, while the largest millisecond bin's share of RTT-18 packets rises from 1.5625% to 12.5%. Faster completion and more concentrated sending are distinct effects. Span alone does not describe the internal density, so both quantities are needed when evaluating smoothing.

7 Loss-free congestion avoidance

ACK count and elapsed time. In the real-valued full-packet form of RFC 5681, Section 3.1, equation (3) [1], let u_k be the window in MSS after CA ACK k, with $u_0 = a \geq 1$. Then

$$\begin{aligned} u_{k+1} &= u_k + \frac{1}{u_k}, \\ u_K^2 &= a^2 + 2K + \sum_{i=0}^{K-1} \frac{1}{u_i^2}. \end{aligned} \tag{14}$$

The identity follows by squaring each update. Since $u_i^2 \geq a^2 + 2i$, it gives $u_K^2 = a^2 + 2K + O(\ln(K+1))$. Gain does not enter this ACK-indexed recurrence. The window at time t is $Mu_{K(t)}$, where $K(t)$ counts actual returning ACKs; substituting t/R for K would discard the many ACKs within an RTT.

What the retained-tail model proves. Extend the ideal scheduler of Section 3 with whole-packet credit $F + Q = \lfloor u \rfloor$. After ACK k it reserves

$$\begin{aligned} m_k &= 1 + \lfloor u_k \rfloor - \lfloor u_{k-1} \rfloor, \\ T_k &= \max(a_k, T_{k-1}) + \frac{m_k}{g u_k}. \end{aligned} \tag{15}$$

Here time is normalized by R, a_k is the corresponding packet's send time plus one, and T_k retains the trailing interval. Put $\mathcal{I}_K = \sum_{k=1}^{K} [a_k - T_{k-1}]_+$. Telescoping (15) splits service into $\sum 1/u_k$ for replacement packets and $\sum(\lfloor u_k \rfloor - \lfloor u_{k-1} \rfloor)/u_k$ for extra whole packets. For fixed $g \geq 1$ and indefinitely continued CA,

$$T_K = \mathcal{I}_K + \frac{u_K + \ln u_K}{g} + C + O(1/u_K). \tag{16}$$

Indeed, $1/u_k = (u_k - u_{k-1}) - 1/(u_{k-1}^2 u_k)$ makes the first sum $u_K + C_1 + O(1/u_K)$. Each sufficiently large integer threshold n is crossed at a window $v_n = n + O(1/n)$, so the second sum is harmonic: $\ln u_K + C_2 + O(1/u_K)$. This proves (16), but does not determine $\mathcal{I}_K$ or a closed window law in time. It applies to the retained-tail model, not directly to SCP's empty-queue reset.

Integer-byte updates in SCP. A new campaign uses Table 1's conditions but sets both initial cwnd and ssthresh to M. This artificial initial state isolates CA; it is not ordinary TCP startup. The research target changes threshold initialization while reusing the shared core, virtual network, and pacing rules. For an ACK covering N_k new bytes, the core uses

$$C_k = C_{k-1} + \max\left(1, \left\lfloor \frac{MN_k}{C_{k-1}} \right\rfloor\right), \qquad C_0 = M. \tag{17}$$

This differs from the real-valued recurrence, even for full-MSS ACKs. RFC 5681 also discusses the integer-zero increment and rounding it to one byte; its equation (3) is one allowed CA method, not every implementation of Reno. For full packets, once $C > M^2/2 = 1{,}022{,}450$ bytes, (17) adds exactly one byte per ACK. Discarded fractional increments accumulate into a measurable difference.

Table 3: CA completion and observed within-run span plateaus.

Target gain	Last ACK (t/R)	$L_{100}, \ldots, L_{1200}$ (ms)
1	1339.97	99
g_c	1339.40	99
2	1339.27	67
4	1339.13	31
8	1339.06	13

All 66 CA transfers completed with matching checksums, exactly 100 ms feedback, no retransmission, and no cap contact. Every observed ACK update matched (17) with zero byte error. At full-packet ACK $K = 750868$, the actual window was 1,476,947 bytes, versus 1,752,403.2934 bytes from (14): measured minus reference, divided by reference, is −15.719%. The final 584-byte ACK then raises the actual window to 1,476,948 bytes and is excluded from this full-packet comparison.

At the same full-packet ACK count, the verified integer window sequence is identical across gains; gain changes when those ACKs occur. Among the five gains in Figure 3, the largest RTT-boundary window difference from gain 1 is 0.629 MSS. Completion changes little, while transmission spans differ strongly (Table 3). Each reported span is unchanged in all 1101 full intervals $n = 100, \ldots, 1200$, within each run. These are finite observations on a millisecond grid. They neither prove infinite periodicity nor make the slow-start reference value $2 \ln 2$ a CA transition.

8 Interpretation, scope, and reproducibility

What the transition explains. The exact reference gives a simple mechanism: when a group's required pacing time exceeds an RTT, reservations accumulate and the window multiplier is below two. Once it fits, ACK availability limits further growth and additional gain chiefly compresses transmission times. The finite threshold is $2D(2^m)$ and approaches $2 \ln 2$; it is not equal to that limit for every group. The instantaneous-rate model reaches the same limiting value through the flight constraint, despite predicting a different low-gain exponent.

What the SCP measurements establish. The native-core experiment reproduces the queued model's low-gain multiplier very closely and exhibits a nearby change to doubled phase patterns. It also shows a finite plateau of fixed-interval spans. The sampled gain $g_c - 0.10$ is still below a doubling multiplier, while $g_c - 0.05$ already doubles. Thus this campaign does not identify $2 \ln 2$ as the

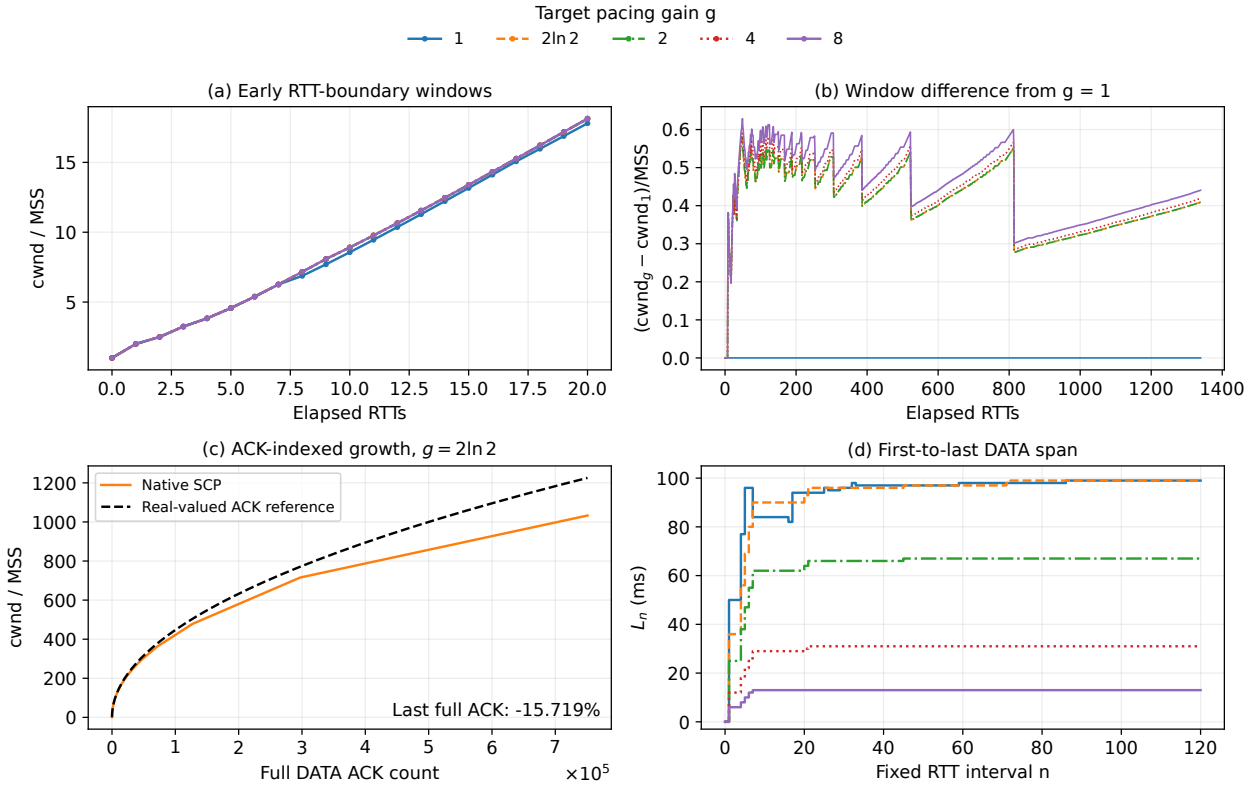


Figure 3: CA from one MSS, five representative gains. (a) Observed RTT-boundary windows. (b) Differences from gain 1. (c) Native critical-target windows and the real-valued reference at matching ACK counts, sampled at RTT boundaries plus the last full-packet ACK. (d) Actual fixed-interval spans, excluding the final-send interval. Lines connect observations; none are fitted.

implementation's exact critical gain. At the ideal critical gain and group size $q = 131072$, (8) gives approximately 99.99945 ms; the implementation's late fixed-interval span is 96 ms. Different scheduling rules and different group definitions must both be accounted for before interpreting that difference as an approximation error.

Relation to existing work. Reno's slow-start update is specified in RFC 5681 [1]. Pacing's interaction with TCP dynamics has been studied well before this work [2]. RFC 9002 gives a cwnd/RTT-based pacing example with a multiplicative factor [3], and the ICCRG pacing draft discusses practical pacing behavior, including compensation for slow-start growth [4]. The contribution here is a compact event-level solution for a particular reservation rule, together with a reproducible examination of how an actual integer implementation changes its predictions. It is not a claim to have first observed that insufficient pacing rate can throttle slow start.

Limits. The study isolates one flow, an initial window of one packet, fixed complete feedback delay, and separate loss-free SS and CA phases. The virtual network intentionally has no bottleneck serialization, propagation variation, cross traffic, delayed ACK policy, or receiver pressure. It therefore measures feedback and scheduling dynamics rather than performance on a real path. The sender executes genuine SCP code, but the environment remains an idealized virtual network. A different initial window, timer granularity, queue policy, or ACK policy can change finite behavior. No conclusion here establishes an optimal Internet pacing gain, an improvement in fairness, or a universal CA window law in time.

Reproduction.

SS uses Actions run 38893710287, source `70f170e`, and its research-branch guide. CA uses successful run 39122340171, source `2b48f8f`, and a separate guide and plotting script. Shared files match baseline `b591e9c`, except the declared CA initialization change. Per-gain artifacts preserve native logs, identities and audits; the CA summary contains tables and figures. Artifacts expire, so the guides supply rerun commands and plotting scripts. Figures preserve the first-send origin without fitting or smoothing. Repetitions are deterministic checks; full-packet comparisons exclude the short ACK.

9 Conclusion

The ACK update alone does not determine growth in time. The immutable-reservation SS model has an exact gain-dependent solution; native transfers closely match its low-gain exponent while differing in phase and finite transition. In CA, gain changes ACK timing while integer-byte updates produce a substantial deficit against the real-valued ACK reference. In both campaigns, higher gain chiefly concentrates transmissions with little completion-time benefit. Evaluating pacing requires the window update rule, its integer representation, and actual packet timing.

References

[1] M. Allman, V. Paxson, and E. Blanton. *TCP Congestion Control.* RFC 5681, 2009. https://www.rfc-editor.org/rfc/rfc5681.

[2] A. Aggarwal, S. Savage, and T. Anderson. Understanding the performance of TCP pacing. *IEEE INFOCOM*, 2000, pp. 1157–1165.

[3] J. Iyengar and I. Swett. *QUIC Loss Detection and Congestion Control.* RFC 9002, Section 7.7, 2021. https://www.rfc-editor.org/rfc/rfc9002.

[4] M. Welzl et al. *Pacing in Transport Protocols.* Internet-Draft, draft-irtf-iccrg-pacing-02, 2026. Work in progress. https://datatracker.ietf.org/doc/html/draft-irtf-iccrg-pacing-02.

A Packet-Time Comparison and the Burst Limit

This appendix supplies the lower-comparison argument for the reference scheduler in Section 3. It does not transfer that theorem to SCP's empty-queue-reset scheduler. Both comparison systems use the same fixed feedback delay and one-MSS increase per ACK.

A.1 The Fixed-Batch Comparator Respects Flight

Let the preceding and current batch sizes be $p = F_{n+1}$ and $q = F_{n+2}$ in round $n \geq 1$. After phase $0 \leq u < 1$, its flight is

$$F(n+u) = p + \lfloor qu \rfloor - \lfloor pu \rfloor \leq q. \tag{18}$$

Its current window is at least q, so the comparator respects the actual-flight limit. It does not reserve all future flight positions at planning time: it can rely on ACKs arriving during the batch. This distinguishes its planning rule from the reference scheduler's invariant $F + Q = w$.

A.2 Every Reference Packet Leaves No Later at Unit Gain

At $g = 1$, $2D(q) \geq 1$ for all $q \geq 1$. Equation (7) reduces to $b_k = 2H_k - 1$, giving

$$s_0 = 0, \qquad s_{2k-1} = 2H_k - 1, \qquad s_{2k} = 2H_k - 1 + \frac{1}{k+1}. \tag{19}$$

The consecutive spacings decrease with packet index:

$$s_{j+1} - s_j = \frac{1}{\lfloor (j+1)/2 \rfloor + 1}, \qquad j \geq 0. \tag{20}$$

The fixed batch of size q starts with packet index $J_n = p + q - 2$. Its timestamps are

$$\widehat{s}_j = n + \frac{j - J_n}{q}, \qquad J_n \leq j < J_n + q. \tag{21}$$

Within this batch, the successive differences of $s_j - \widehat{s}_j$ are the decreasing spacings above minus $1/q$. A maximum is attained at $j_* = 2q-3$. This index belongs to the batch because $j_* - J_n = q-p-1 \geq 0$ and $J_n + q - 1 - j_* = p \geq 1$. Substitution yields

$$\max_{J_n \leq j < J_n + q} (s_j - \widehat{s}_j) = -E_n, \qquad E_n = n + 2 - \frac{p-1}{q} - 2H_q. \tag{22}$$

Initially $p = 1, q = 2$, and $E_1 = 0$. In the next batch, (p, q) becomes $(q, p+q)$, so

$$\begin{aligned} E_{n+1} - E_n &= 1 - \frac{q-1}{p+q} + \frac{p-1}{q} - 2\sum_{i=1}^{p} \frac{1}{q+i} \\ &= 2\sum_{i=1}^{p} \frac{i(p-i)}{q(q+p)(q+i)} \geq 0. \end{aligned} \tag{23}$$

For the second equality, subtract $1/(q+i)$ from $(q+p-i)/[q(q+p)]$; the difference is $i(p-i)/[q(q+p)(q+i)]$. Summing these identities gives the second line of (23). Thus $E_n \geq 0$, proving $s_j \leq \widehat{s}_j$ in every batch. Packet zero is common to both systems. Earlier corresponding sends give earlier corresponding ACKs, hence no smaller window at any observation time.

A.3 Extension to Larger Gains

For $g_2 \geq g_1 \geq 1$, induction over ACK batches gives $s_j(g_2) \leq s_j(g_1)$. The initial send is common. If earlier packet times are ordered, their fixed-delay ACK times are ordered. The previous reservation tail is also ordered, since every newly added spacing decreases as gain increases. Taking the maximum of the ACK time and the previous tail preserves the order, and so does adding the second packet's spacing.

Combining this monotonicity with the unit-gain comparison and the flight upper bound proves, specifically for (3),

$$MF_{n+2} \leq \mathrm{cwnd}_g(nR) \leq M2^n, \qquad g \geq 1. \tag{24}$$

The actual-time comparison is essential. Merely appending more packets earlier does not order their departures under arbitrary queue or timestamp rules.

A.4 Approaching Burst Sending

For $g \geq g_c$, $B_m = m + 1$. For any fixed packet index j in group m, its timestamp therefore has the form $s_j = m + 1 + d_j/g$, with $d_j \geq 0$ independent of gain. Including packet zero,

$$\lim_{g \to \infty} s_j(g) = \lfloor \log_2(j+1) \rfloor. \tag{25}$$

This is the immediate-send sequence: one packet at time zero, two at one RTT, four at two RTTs, and so on. For finite gain, packets remain locally spaced, yet their group occupies a shrinking fraction of an RTT. Uniform spacing within a group therefore coexists with growing concentration across time.

There is a boundary subtlety. At finite high gain, $W_n = 1 + 2^{n-1}$, whereas the immediate-send window is 2^n. ACKs just after a boundary coalesce onto that boundary as gain tends to infinity. Counting events at a jump is not continuous under convergence of their timestamps. The packet-time burst limit is valid without asserting pointwise convergence of the window at every integer RTT.